

1024-bit HDC Accelerator on Zynq-7020

Slide-by-slide explainer for `HDC_Research_Review.pptx`

What each slide shows · how to present it · questions the professor may ask (with answers)

Companion to the 22-slide deck · SystemVerilog + Python golden reference

Platform: Xilinx Zynq-7020 (ZedBoard) · Vivado 2024.2 · Prepared July 2026

How to use this document. Each numbered section below corresponds to one slide in `HDC_Research_Review.pptx`, in order. For every slide you get: *What's on the slide* (so you can glance and orient), *How to explain it* (the narration / talking points), and *Likely questions* the professor may ask with ready answers. Keep the deck open beside this; the section numbers match the slide numbers. If time is short, lead with slides 15–19 (the accuracy story and the research contribution).

SLIDE INDEX

- | | |
|------------------------------------|------------------------------------|
| 1 Title | 12 Verification coverage |
| 2 The problem & motivation | 13 Board results — 3 paths |
| 3 What is HDC? | 14 Resources & the D-sweep |
| 4 The four HDC primitives | 15 The two-baseline accuracy story |
| 5 Application — EMG gestures | 16 Why 74% is not a weakness |
| 6 System architecture — PS + PL | 17 Hook A — 3-axis Pareto |
| 7 The encoding datapath | 18 Twist 1 & Twist 2 |
| 8 Module hierarchy | 19 Comparison baselines |
| 9 Computational microarchitecture | 20 Status & roadmap |
| 10 Cycle-level behaviour / FSM | 21 Summary & next steps |
| 11 Verification — golden reference | 22 Thank you / Q&A |

Slide 1 — Title

WHAT'S ON THE SLIDE

Project title (*1024-bit Hyperdimensional Computing Accelerator on Zynq-7020*), the subtitle "streaming, bit-exact-verified HDC classifier for EMG hand-gesture recognition", target venue **DATE 2027**, and the platform line (ZedBoard, SystemVerilog + Python golden reference).

HOW TO EXPLAIN IT

"This is a hardware accelerator for Hyperdimensional Computing, built and verified on a Zynq FPGA. The two words that define the work are **streaming** (data flows through at inference rate over DMA) and **bit-exact-verified** (the RTL matches a Python golden model exactly). The application is EMG hand-gesture recognition, and I'm targeting DATE 2027."

LIKELY QUESTIONS

Q What is the one-sentence contribution?

A A measured accuracy/energy/area Pareto for a streaming 1024-bit HDC core on Zynq, plus an informed-vs-random bit-pruning result — not a re-port of prior HDC accuracy.

Q Why Zyng / ZedBoard specifically?

A It pairs an ARM CPU (PS) with FPGA fabric (PL) on one chip, so I can stream data from software to hardware over AXI-DMA and also run a fair ARM-only software baseline on the same silicon.

Slide 2 — The problem & motivation

WHAT'S ON THE SLIDE

Three bullets: edge inference must be cheap/low-power; HDC is a bitwise, brain-inspired alternative to multiply-heavy CNN/MLP; the application is EMG (Rahimi et al. benchmark). A blue box states the **thesis**: a streaming 1024-bit HDC core with zero DSP/BRAM, where per-bit pruning cuts energy — the contribution being a measured Pareto.

HOW TO EXPLAIN IT

"Always-on edge devices — prosthetics, wearables — can't afford GPU-style multiply-accumulate. HDC replaces that with bitwise operations on long binary vectors, which map beautifully to FPGA logic. I use the EMG gesture dataset from Rahimi's 2016 paper because it's the standard HDC-on-EMG benchmark, so I can compare directly with the literature."

LIKELY QUESTIONS

Q Why not just use a small neural network?

A We measured one — a tiny int8 MLP hits ~93% (slide 19). HDC's argument isn't peak accuracy, it's efficiency: bitwise ops, one-shot training, zero DSP, and tolerance to bit errors that lets us prune aggressively. That trade-off is exactly what the Pareto quantifies.

Q Is HDC actually used in industry, or is it academic?

A It's still mostly research, but its appeal for ultra-low-power always-on sensing (biosignals, IoT) is real. The point of this work is to give hardware designers measured numbers on the energy/accuracy trade-off, which is what's missing.

Slide 3 — What is Hyperdimensional Computing?

WHAT'S ON THE SLIDE

Four bullets: every concept becomes one 1024-bit hypervector; random vectors are near-orthogonal in high dimensions (capacity + noise tolerance); classification is nearest-prototype by Hamming distance; the model is Binary Spatter Code (BSC). A green box lists why it suits FPGAs.

HOW TO EXPLAIN IT

"The core idea: in 1024-dimensional binary space, two random vectors differ in about half their bits, so they're 'nearly orthogonal'. That gives huge representational capacity and means flipping a few bits barely changes a vector's identity — inherent error tolerance. Classifying is just finding the nearest stored vector by Hamming distance. I use Binary Spatter Code: binary vectors, XOR binding — the most hardware-friendly flavour."

LIKELY QUESTIONS

Q Why 1024 dimensions — why not more or fewer?

A 1024 is the sweet spot for this task and it's a clean 16×64-bit word layout for the datapath. But D is a parameter — the D-sweep (slide 14) measures 256 to 2048 so the dimension choice is itself a result, not an assumption.

Q Where does the noise tolerance actually help you?

A It's what makes bit-pruning viable: I can zero out a large fraction of bits in the distance computation and still classify correctly, because the class signal is spread across many bits. That's the basis of Hook A.

Slide 4 — The four HDC primitives

WHAT'S ON THE SLIDE

A table mapping each primitive to its math, meaning, and RTL block: **Bind** (XOR → `xor_permute_top`), **Permute** (cyclic shift → `permute_stage`), **Bundle** (majority → `bundle_unit`), **Search** (argmin Hamming → `popcount_am`).

HOW TO EXPLAIN IT

"All of HDC is four operations, and each one is a verified RTL block in my design. Bind (XOR) associates two concepts, like a channel with its value. Permute (a bit rotation) encodes position. Bundle (bitwise majority vote) superposes many vectors into one. Search finds the nearest class prototype. Encoding uses the first three; search does the classification. Every one is XOR, shift, or popcount — no multipliers."

LIKELY QUESTIONS

Q Why is XOR the right binding operation?

A XOR is its own inverse and produces a result dissimilar to both inputs, which is exactly the algebra binding needs — you can unbind by XORing again, and bound pairs don't collide with their parts.

Q Does the majority vote need a tie-break?

A Yes — for an even number of bundled vectors, ties are broken deterministically (matching the Python golden), so RTL and reference agree bit-for-bit.

Slide 5 — Application: EMG hand-gesture recognition

WHAT'S ON THE SLIDE

The task (4-channel forearm EMG → gestures, for prosthetics) and the 5-step window-to-class pipeline: capture window → quantise features to 16 levels → look up item memories → bind/permute/bundle → masked Hamming search vs 5 prototypes. A box explains that "training" in HDC is just bundling.

HOW TO EXPLAIN IT

"A short window of the four EMG channels is captured, features are quantised into 16 levels, and each (channel, feature, value) triple indexes a hypervector in an item memory. Those are bound, permuted, and bundled into one query vector, which is compared to the five gesture prototypes. Training is not gradient descent — for each gesture I just bundle its training windows into one prototype. That single-pass training is a big part of HDC's appeal."

LIKELY QUESTIONS

Q How are the item-memory hypervectors generated?

A Random seed-42 vectors (committed, reproducible). They're fixed ROMs in hardware — the same seed is used in the Python golden so the two match exactly.

Q Only 5 gestures / 5 subjects — is that enough?

A Five classes is the standard benchmark configuration. Five subjects is the current export; Twist 2 needs a larger 36-subject export, which is flagged as a known dependency (slides 18, 20).

Slide 6 — System architecture: Zynq PS + PL

WHAT'S ON THE SLIDE

The [arch.svg](#) block diagram: PS (ARM) streams EMG windows from DDR over AXI-DMA to the PL HDC core, which encodes, classifies, and returns labels. Control is AXI4-Lite; data is AXI4-Stream.

HOW TO EXPLAIN IT

"The Zynq is a system-on-chip: hardened ARM (the PS) plus FPGA fabric (the PL). My HDC core lives in the PL. The ARM sets things up over AXI4-Lite — loads prototypes and the mask, starts the core — then a DMA engine streams windows into the core over AXI4-Stream and results come back. This split is what makes the throughput and the fair software comparison possible."

LIKELY QUESTIONS

Q Why separate control (AXI-Lite) from data (AXI-Stream)?

A Control is occasional, register-mapped, latency-insensitive; data is high-rate and throughput-critical. Streaming avoids per-window register writes, which is what gets us to ~216k windows/s.

Q Is the ARM a bottleneck in the streaming path?

A No — once DMA is running, the ARM just manages descriptors. The batch/scatter-gather path (slide 13) keeps the core fed back-to-back.

Slide 7 — The encoding datapath

WHAT'S ON THE SLIDE

The `pipeline.svg` figure showing Eq. (3.1): a 4-channel × 5-feature grid (20 binds), position-permuted, bundled into the query HV, with the new `pruning_mask` gating which bits count in the associative-memory distance.

HOW TO EXPLAIN IT

"This is the encoder from the research plan, Equation 3.1. Twenty (channel, feature) pairs are each bound to an item-memory vector, permuted by feature position, and all twenty are bundled by majority into one 1024-bit query. The orange block is the pruning mask — it decides which bit positions are counted in the Hamming distance, and that's the lever for the energy study."

LIKELY QUESTIONS

Q Why $4 \times 5 = 20$ binds — where does the 5 come from?

A Five features per channel in the Eq. (3.1) encoding. It's the hardware-faithful encoder; it differs from Rahimi's spatial-record encoding, which is the root of the 74-vs-90 accuracy discussion (slide 15).

Q Does the mask change the encoder output or only the search?

A Only the search — the query HV is unchanged; the mask zeroes gated bits before popcount in the AM, so pruning removes switching activity without touching the encode.

Slide 8 — Module hierarchy

WHAT'S ON THE SLIDE

The `hierarchy.svg` tree of RTL modules. A green box highlights that `pruning_mask.sv` is newly extracted as a dedicated module (research-plan §5.3.3), holding the D-bit mask and gating Hamming distance, verified bit-exact (129/129 checks PASS).

HOW TO EXPLAIN IT

"This shows how the blocks compose — encoder and associative memory under the core, wrapped by the AXI-Lite and stream interfaces. The thing to note is that I pulled the pruning mask out into its own named, parameterised module this cycle. Previously it was buried inside the popcount unit; now it's a clean block that supports both an AXI word-write port and a full-width parallel load, so the existing verified board path is untouched. It passes 129 of 129 checks."

LIKELY QUESTIONS

Q Why extract the mask into its own module now?

A So the pruning study (Hook A) has a clean, independently verifiable interface, and so I can load different masks at runtime without rebuilding the bitstream. It also matches the research-plan module map.

Q What are the "129 checks"?

A The `run_pruning_mask_cosim` harness — full-mask loads plus AXI word-addressed writes compared bit-for-bit against the Python model.

Slide 9 — Computational microarchitecture (word-serial)

WHAT'S ON THE SLIDE

The `compute_datapath.svg` : the 1024-bit HV is stored as 16×64-bit words; the AM streams one 64-bit word/cycle — XOR query⊕prototype, AND mask, 64-bit popcount, accumulate distance. A green box explains why word-serial (small popcount + accumulator reuse → 0 DSP, easy timing).

HOW TO EXPLAIN IT

"Rather than a giant 1024-bit comparison in one cycle, I process 64 bits per cycle: XOR the query against a prototype word, AND with the mask word, popcount the result in a LUT adder tree, and accumulate. Sixteen words per class, looped over classes, then argmin. Word-serial means one small popcount and one accumulator are reused — that's how we stay at zero DSP, zero BRAM and close timing at 100 MHz. And because the mask zeroes bits before the popcount, pruning directly removes switching activity — the energy saving is real, not bookkeeping."

LIKELY QUESTIONS

Q Doesn't word-serial hurt throughput vs a parallel tree?

A It trades latency for area and timing, but the pipeline hides it across windows — we still hit ~216k windows/s. A full parallel popcount would blow up LUTs and timing for little end-to-end gain.

Q Why 64-bit words?

A Matches the AXI-Stream data width and gives a clean 16-word HV; the popcount tree for 64 bits fits comfortably in LUTs at 100 MHz.

Slide 10 — Cycle-level behaviour: AM FSM & latency

WHAT'S ON THE SLIDE

The `compute_fsm.svg` and a cycle-count table: encode ~23 cycles, bundle combinational, classify 264 cycles, ≈ 287 cycles/window $\approx 2.9 \mu\text{s}$ @ 100 MHz. A box notes CNT_W and prune ratio are the two micro-architectural knobs of Hook A.

HOW TO EXPLAIN IT

"Here's the cycle budget. Encoding a window is about 23 cycles; the masked-Hamming classify over 8 class slots is 264 cycles; roughly 287 cycles per window, about 2.9 microseconds at 100 MHz, and it pipelines across windows. The two knobs that Hook A turns — the bundle counter width CNT_W and the prune ratio — change the datapath's switching and area *without* changing its structure, so they're clean axes to sweep."

LIKELY QUESTIONS

Q You said 8 classes but only 5 gestures?

A The AM is provisioned for 8 class slots (power-of-two indexing); 5 are used for gestures, the rest are unused. The cycle count reflects the provisioned 8.

Q What does CNT_W actually change physically?

A The width of the 1024 saturating counters in the bundler. Narrower counters save area/energy but can saturate and lose accuracy — which is precisely what the sweep measures.

Slide 11 — Verification: the Python golden reference

WHAT'S ON THE SLIDE

The `verification.svg` flow: a bit-exact Python model (`hdc_ref.py`) is the single source of truth, generating both the stimulus and the expected output so RTL and reference can't drift.

HOW TO EXPLAIN IT

"This is the methodological backbone. There is no hand-written expected output anywhere. The Python model generates the random stimulus *and* computes the expected result; the RTL is fed the same stimulus in co-simulation and must match bit-for-bit. So the reference and the hardware physically cannot silently diverge — any drift fails a test immediately."

LIKELY QUESTIONS

Q How do you know the Python model itself is correct?

A It's validated against the literature: the same model reproduces Rahimi's ~90% (slide 15, Stage A/B). So the golden is anchored to published results, and the RTL is anchored to the golden.

Q Is co-simulation as trustworthy as formal verification?

A It's directed-random with thousands of cases per block and full end-to-end coverage, then confirmed on real silicon over 658k windows. For this datapath that's strong evidence; formal could be a future add for the control FSMs.

Slide 12 — Verification coverage: 7 + 1 harnesses

WHAT'S ON THE SLIDE

A table of the eight co-sim harnesses and case counts (bind/permute 1000, bundle 500, AM 500, encoder 500, core 500, AXI 200, stream 200, pruning-mask 64), all PASS on Questa and Vivado xsim.

HOW TO EXPLAIN IT

"Every block has its own harness, and there's an end-to-end one and an AXI/stream one, plus the new pruning-mask harness. Case counts run up to a thousand. Everything passes on both simulators — Questa and Vivado's xsim — so the result isn't tool-specific."

LIKELY QUESTIONS

Q Why different case counts per harness?

A Larger state spaces get more cases (bind/permute 1000); protocol harnesses (AXI/stream) need fewer because they test sequences, not arithmetic breadth.

Q Do the harnesses cover back-pressure and stalls?

A Yes — `run_stream_cosim` injects random gaps and back-pressure to prove the stream path is robust, not just the happy path.

Slide 13 — Board results: three measurement paths

WHAT'S ON THE SLIDE

KPI cards (74.24% board accuracy, $\Delta 0.00\%$ vs golden over 658k windows, ~216k win/s, 0/0 DSP/BRAM) and a table comparing Phase 1 (AXI-Lite), Phase 2 (DMA stream), Phase 3 (SG batch) on golden, latency, throughput, timing.

HOW TO EXPLAIN IT

"Three ways to drive the *same* core, each a data point for the paper. Phase 1 is register-mapped control — the baseline, 3 μ s/window. Phase 2 is the DMA streaming path. Phase 3 is scatter-gather batch, the throughput story at ~216k windows/s with positive timing slack. All three pass the 200-case golden, and the full EMG replay matches the golden to zero over 658 thousand windows."

LIKELY QUESTIONS

Q Why is Phase 2 slower per window than Phase 1?

A Single-window DMA has descriptor setup overhead; that's why Phase 3 batches many windows through a descriptor ring to amortise it and get the real throughput number.

Q What does WNS +0.111 ns mean and why care?

A Worst negative slack is positive, so the design meets the 100 MHz timing constraint with margin after place-and-route — it's a genuine closed design, not a simulation-only claim.

Slide 14 — Resource utilisation & the D-sweep

June gap CLOSED

WHAT'S ON THE SLIDE

Integrated utilisation (35,206 LUT = 66.2%, 96.3% slices, 0 DSP/BRAM) and the OOC core-only D-sweep table ($D = 256/512/1024/2048 \rightarrow$ LUT, util, WNS, Fmax, cosim PASS). Note: $D=2048 = 111\%$ LUT = a reportable Pareto boundary.

HOW TO EXPLAIN IT

"Two things. First, the full system is pure logic — zero DSP, zero BRAM — with the item memory inferred as LUT ROM. Second, this is the D-axis of the Pareto, now measured: I synthesised the core out-of-context at four dimensions and ran functional co-sim at each. LUTs scale roughly linearly with D. $D=1024$ fits comfortably; $D=2048$ core-only is 111% of the device — it doesn't fit standalone, and that boundary is itself a result worth reporting."

LIKELY QUESTIONS

Q Why is $D=1024$'s WNS the tightest even though it's not the biggest?

A Timing depends on routing/congestion, not just size; $D=1024$ is the shipping config with the full system, so its critical path is the most stressed. All configs still meet 100 MHz.

Q Is out-of-context synthesis a fair area measure?

A It isolates the core's logic cleanly for the D-axis comparison. For the deployable number I also report the full integrated build (66.2% LUT). Both are on the slide on purpose.

Slide 15 — The two-baseline accuracy story

read carefully

WHAT'S ON THE SLIDE

A table of three tracks: Stage A MAP (Python, 90.36%), Stage B BSC (Python, 90.30%), RTL encoder (Python + board, 74.24%). A red "defense one-liner" box.

HOW TO EXPLAIN IT

"This is the slide to be careful and confident on. I report two accuracy numbers deliberately. In Python I reproduce the literature — about 90% — with the same encoding Rahimi used. On the FPGA I run a *different*, hardware-faithful encoder that lands at 74.24%. These are different algorithms, so they were never supposed to match. The deliverable is a verified deployment path plus an energy/pruning study — not a second copy of the 90% number."

Say the one-liner verbatim: "We reproduced ~90% in Python matching Rahimi; the FPGA runs a different, bit-exact-verified encoder at 74.24%; the contribution is measured energy and informed-pruning Pareto on that verified path — not a second accuracy port."

LIKELY QUESTIONS (THIS SLIDE DRAWS THE MOST)

Q Rahimi got ~90%. Why is your hardware only 74%?

A Because it's a different encoder (Eq. 3.1 grid vs Rahimi's spatial records), not a degraded version. I do reproduce ~90% in Python. The board matches its own golden to $\Delta 0.00\%$ over 658k windows, so the system is provably correct; the contribution is relative (pruning, energy), so the absolute number doesn't affect the claims.

Q Then why not put the 90% encoder on the FPGA?

A It's a richer encoding that costs more logic and design time. I locked down a fully verified streaming path first. The research questions don't depend on the absolute encoder, so I can publish on the 74% path now and upgrade the encoder later — it's on the roadmap.

Q Isn't reporting 74% risky for a paper?

A The opposite — the full 74↔90 gap is traced (encoding, item-memory seed, feature handling), so reviewers see an honest, debugged pipeline. The headline results are relative gaps, which are robust.

Slide 16 — Why 74% is not a weakness (six points)

WHAT'S ON THE SLIDE

Six numbered points: (1) ~90% reproduced in Python; (2) 74% is a different encoder; (3) verification fidelity $\Delta 0.00\%$ over 658k; (4) it's a systems study; (5) headline claims are relative; (6) reporting both is a credibility strength.

HOW TO EXPLAIN IT

"This is the backup for the previous slide — six ways to say the same thing so I can meet whichever angle the question comes from. The anchor is point 3: bit-exact fidelity over 658 thousand real windows. Points 4 and 5 are the framing: this is a hardware/energy study and the claims are relative, so 74 vs 90 is simply the wrong axis to judge it on."

LIKELY QUESTIONS

Q Earlier you mentioned 59% — what was that?

A A prototype-training bug: a saturating 6-bit counter clipped when bundling many samples, plus a prototype-load indexing bug. Both fixed and documented; accuracy went to the correct 74.24%. I keep it in the log as evidence of a debugged pipeline.

Q Which of the six is the strongest point?

A Verification fidelity (point 3) — $\Delta 0.00\%$ over 658k windows proves the deployed system is correct, which is the actual engineering deliverable.

Slide 17 — Hook A: 3-axis Pareto

contribution

WHAT'S ON THE SLIDE

The novelty statement (prior work prunes dimension; the open axis is *which* bits) and the axis table: D (DONE, synth+cosim), CNT_W (running, Python sweep), bit-pruning ratio (running, Python sweep).

HOW TO EXPLAIN IT

"This is the main contribution. Existing FPGA-HDC work prunes the dimension D. The under-explored question is not how many bits but *which* bits. Hook A sweeps three knobs — D, the bundle precision CNT_W, and the bit-pruning ratio — and maps the accuracy/energy/area surface. The D-axis is measured on silicon; the CNT_W and pruning axes are running now as an automated 64-cell Python sweep, with the quick-sanity run already passing. INA219 will add the measured energy axis."

LIKELY QUESTIONS

Q What exactly is the 64-cell grid?

A 4 dimensions × 4 CNT_W values × 4 keep-ratios = 64 configurations, each evaluated over 5 subjects on the RTL-matched encoder with Fisher-informed masks. Output lands in `results/hook_a/`.

Q How long does the sweep take and is it done?

A ~45–50 h single-core on the VDI; it was ~26% through (up to D=512) at last check, with a checkpointed partial JSON so I can preview the Pareto before it finishes.

Q How do you get energy without running every cell on hardware?

A Accuracy comes from the Python sweep, area from the D-sweep synthesis, and energy from INA219 on a few representative board anchors — I don't need to synthesise all 64.

Slide 18 — Twist 1 & Twist 2

WHAT'S ON THE SLIDE

Twist 1: informed (Fisher-ratio) mask vs random mask at equal density; claim ≥ 5 pp gap at iso-density.

Twist 2: cross-subject mask transfer; either outcome publishable (needs 36-subject export; currently 5).

HOW TO EXPLAIN IT

"Two focused experiments on top of Hook A. Twist 1 is the headline: at the same bit-count, does keeping the most discriminative bits — ranked by Fisher ratio, between-class over within-class variance — beat a random mask? I expect at least a 5-point accuracy gap, which proves bit *position* carries information. Twist 2 asks whether a mask trained on some subjects transfers to unseen ones. If it does, there's a universal bit-importance map; if it doesn't, that motivates on-device adaptation — either way it's a result."

LIKELY QUESTIONS

Q Why would informed pruning beat random?

A Because hypervector bits aren't equally informative. Fisher ranks bits by class separability; keeping high-ranked bits preserves signal while dropping noise-like bits. Random drops both alike. The gap quantifies how much structure the encoding carries per bit.

Q What if Twist 2 fails to transfer?

A That's still publishable — it means EMG bit-importance is subject-specific and argues for lightweight on-device mask adaptation. I framed it so either outcome is a contribution.

Q Why do you need 36 subjects for Twist 2 but not Twist 1?

A Twist 1 is a within-run comparison at one config, fine on 5 subjects. Cross-subject transfer needs enough held-out subjects to be statistically meaningful, hence the larger export.

Slide 19 — Comparison baselines

accuracy done

WHAT'S ON THE SLIDE

Table: PL DMA (this work) 74.24% @ ~4 μ s; ARM-only HDC 74.15% @ 818 μ s (1,222 win/s), 200/200 golden; tiny int8 MLP 93.01% fp / 92.99% int8; AXI-Lite path ~3 μ s. Green box: PL DMA ~200 $\times$ faster than ARM at matched accuracy; INA219 adds energy.

HOW TO EXPLAIN IT

"To make the efficiency argument fair I built the obvious alternatives. The *same* HDC algorithm in C on the ARM core hits essentially the same accuracy (74.15 vs 74.24) but takes 818 microseconds per window — the FPGA path is about 200 $\times$ faster. And a tiny int8 MLP reaches 93%, which is the honest 'a small NN is more accurate' point — but it doesn't have HDC's bitwise, zero-DSP, prune-friendly profile. Accuracy for all baselines is done and committed; the energy column comes from INA219 next."

LIKELY QUESTIONS

Q Is the 200 $\times$ speedup a fair comparison?

A Yes — identical algorithm, identical protocol, same silicon (Cortex-A9 vs PL on one Zynq), and the ARM version passes the same 200-case golden. It isolates the hardware acceleration.

Q The MLP is more accurate — doesn't that undercut HDC?

A No — it sets the accuracy reference honestly. HDC's case is efficiency and robustness, which the energy Pareto will quantify. Reporting the MLP strengthens credibility rather than hiding the trade-off.

Q Why is int8 MLP accuracy basically equal to float here?

A Post-training int8 quantisation on a small model with this feature set barely moves accuracy (93.01 $\rightarrow$ 92.99%); the drop would grow on harder tasks. The number is measured over the full 5-subject TEST split.

Slide 20 — Status & roadmap

WHAT'S ON THE SLIDE

The `roadmap.svg` gantt: *Done* — golden + baselines, RTL + cosim, pruning_mask + D-sweep, Phases 1–3, EMG replay, comparison baselines; *Next* — INA219 energy, Hook A sweep, Twist 1/2; *Paper* — figures + DATE draft.

HOW TO EXPLAIN IT

"Where things stand: all bring-up, verification, the D-sweep, and the comparison baselines are done and ahead of the plan. The active items are the INA219 energy measurement and the running Hook A sweep, then the two twists, then the write-up. The hard infrastructure risk is retired — what's left is measurement science and the paper."

LIKELY QUESTIONS

Q Are you on track for DATE 2027?

A Yes — the engineering is complete and ahead of schedule; the remaining Aug/Sep blocks are measurement and writing, which are lower-risk.

Q What's the single biggest risk right now?

A Two small ones: the INA219 hardware needs to arrive and be wired, and Twist 2 needs the 36-subject export. Neither blocks the headline Hook A / Twist 1 results.

Slide 21 — Summary & next steps

WHAT'S ON THE SLIDE

"Done & verified" list (RTL + 8 harnesses, Phases 1–3 + EMG replay, pruning_mask + D-sweep, Python ~90% baselines, comparison baselines) and the critical path: INA219 energy → Hook A sweep → Twist 1/2 → paper.

HOW TO EXPLAIN IT

"To wrap: the hardware is done and provably correct, the D-axis and the baselines are measured, and the research machinery — the pruning mask and the sweep — is in place and running. The critical path to the paper is short: energy measurement unblocks every Pareto plot, the sweep finishes the accuracy grid, the twists give the headline figures, and then it's writing."

LIKELY QUESTIONS

Q If you could only finish one thing next, what is it?

A INA219 energy — it unblocks the energy axis of every Pareto figure and the ARM-vs-PL efficiency claim, which are the paper's core.

Q What can proceed in parallel with the sweep?

A The energy measurement (board + I²C), a single-point Twist 1 run, and a Pareto preview from the checkpointed partial results — none of those wait on the full grid.

Slide 22 — Thank you / Q&A

WHAT'S ON THE SLIDE

Closing lead slide: hardware is done and provably correct (bit-exact on 658k windows); 74% is a faithful hardware encoder; the research is the energy/accuracy Pareto + informed-pruning result.

HOW TO EXPLAIN IT

"Close on the three sentences that survive if they remember nothing else: the hardware works and is bit-exact over 658 thousand windows; 74% is a faithful encoder, not a failure, and I already match ~90% in Python; and the contribution is the energy/accuracy Pareto plus the informed-pruning result — which stand regardless of the absolute accuracy."

LIKELY CLOSING QUESTIONS

Q What would make this a strong DATE paper specifically?

A A clean Pareto with measured silicon energy, the ≥ 5 pp informed-vs-random result, and the zero-DSP/200 $\times$ -vs-ARM efficiency story — a hardware contribution with real measurements, not just accuracy.

Q What's the next milestone I'll see from you?

A The completed Hook A sweep with a first Pareto figure, and the INA219 energy-per-window numbers for the PL and ARM paths.